

Subject:

# PHP with MySQL

**BCA 5th Semester**

Course Code: BCAC501

Comprehensive notes provided by

**Great Heights**

Build | Believe | Become



## Introduction to Web Development

Web development is the process of building, creating, and maintaining websites or web applications for the internet or an intranet. It involves multiple disciplines, including web design, web publishing, web programming, and database management. Over time, it has evolved from simple static HTML pages to complex, interactive, and data-driven applications.

Web development mainly consists of: Frontend + Backend + Database

Frontend development focuses on the client-side—everything users see and interact with in their browsers. Core technologies include:

- HTML: Structures the content of web pages.
- CSS: Styles and formats the layout, colors, and fonts.
- JavaScript: Adds interactivity, animations, and dynamic content updates.

Example :

```
<!DOCTYPE html>
<html>
<head>
<title>My First Web Page</title>
<style>
  body { font-family: Arial; background-color: #f4f4f4; }
  h1 { color: #333; }
</style>
</head>
<body>
<h1>Hello, World!</h1>
<p>This is a basic HTML page with CSS styling.</p>
<script>
  alert("Welcome to my website!");
</script>
</body>
</html>
```

Backend development handles the server-side—processing requests, managing databases, and controlling application logic. Common backend languages and frameworks include Node.js, Python (Django/Flask), PHP (Laravel), Java (Spring Boot), and Ruby on Rails. Databases can be SQL (MySQL, PostgreSQL) or NoSQL (MongoDB, Redis) depending on the data structure and scalability needs.

APIs (REST, GraphQL) enable communication between frontend and backend, often using JSON or XML for data exchange.

Version control systems like Git help manage code changes, while deployment involves hosting the application on platforms such as AWS, Azure, or Firebase.

Database is an organized collection of data that allows information to be stored, managed, searched, updated, and retrieved efficiently. In a web application, a database stores important information such as user details, login credentials, products, orders, student records, marks, etc. Instead of keeping data in separate files, a database organizes it in a structured manner, usually using tables, rows, and columns. In our PHP with MySQL course, MySQL is used as the database system, while PHP communicates with MySQL to insert, retrieve, update, and delete data.

Example: A student database may contain a Students table with columns such as Student\_ID, Name, Email, Course, and Marks.

## How Does a Website Actually Work?

When a user makes a request on a website, several parts of the application work together. These include: User interface, Middleware systems, Databases, Servers, and Browser.

Suppose a student types: `www.example.com`

into the browser.

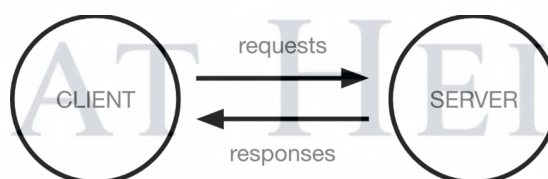
**Step 1** — Browser sends a request

The browser sends a request to the web server.

**Step 2** — Server processes the request

The server receives the request and determines what needs to be returned.

If it is a PHP application, the PHP code may need to execute.



**Step 3** — Server sends a response

The server sends the result back to the browser.

**Step 4** — Browser displays the page

The browser receives HTML, CSS, JavaScript, images, etc., and renders the webpage.

Example:

Think about ordering food online. The user opens a food-delivery website.

User → Website → Restaurant Server → Database

1. User searches: "Chicken Biryani"
2. The website sends the request to the server.
3. The server asks the database: "Give me restaurants/products matching Chicken Biryani."
4. The database returns the information.
5. The server sends it back to the user's browser.

## Client and Server

**Client:** A client is a device or application that requests services or resources from a server.

Client → Request → Server

- Clients can be various devices such as smartphones, desktops, laptops, or tablets.
- The primary function of a client is to initiate communication with a server by sending requests for data or services.
- Clients are typically designed to be user-friendly and provide interfaces for users to interact with applications or access information.
- Clients are often lightweight in terms of processing power and storage capacity compared to servers. They rely on servers to perform complex tasks or store large amounts of data.
- Examples of clients include web browsers, email clients, and FTP clients.
- Clients can communicate with multiple servers simultaneously, depending on the requirements of the user or application.

**Server:** A server is a device or application that provides services or resources to clients over a network.

User's Browser → Request → Web Server → Response → User's Browser

- Servers are typically more powerful in terms of processing capabilities and storage capacity compared to clients.
- The primary function of a server is to wait for incoming requests from clients and respond by providing the requested data or services. Servers are designed to handle multiple client requests simultaneously, known as concurrency.
- They can process and respond to numerous requests in parallel, ensuring efficient communication with clients.
- Servers are responsible for storing and managing data, performing computations, and executing tasks on behalf of clients.
- Examples of servers include web servers, email servers, and file servers.

## Client-Side vs. Server-Side

| Feature             | Client-Side                                       | Server-Side                                             |
|---------------------|---------------------------------------------------|---------------------------------------------------------|
| Meaning             | Code that runs on the user's device/browser       | Code that runs on the web server                        |
| Execution           | Executed by the browser                           | Executed by the server                                  |
| Main purpose        | Handles webpage appearances and user interactions | Handles business logic, data processing, authentication |
| Common Technologies | HTML, CSS, JavaScript                             | PHP, Python, Java, Node.js, etc.                        |
| Database access     | Generally does not directly access the database   | Can communicate with databases                          |
| Speed               | Usually fast because it runs locally              | Depends on server processing and network                |
| Visibility          | Code is generally accessible by the user          | Source code is generally hidden from the user           |
| Security            | Less suitable for sensitive operations            | Better suited for sensitive operations                  |
| Example             | Form validation, animations, button interactions  | Login verification, database operations                 |
| Runs on             | User's computer/mobile                            | Web server                                              |

## HTML Basics

HTML (HyperText Markup Language) is the standard language for creating web pages. Every HTML document follows a specific structure that browsers use to render content.

- HTML = HyperText Markup Language
- HTML is used to define the structure of a webpage.
- HTML is not a programming language. It is a markup language.

BUILD | BELIEVE | BECOME

## Basic HTML Structure

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My First Webpage</title>
</head>
<body>
  <h1>Welcome to My Website</h1>
  <p>This is my first paragraph in HTML.</p>
  <a href="https://www.example.com">Visit Example</a>
  
</body>
</html>
```

## Common HTML Tags

| Tag   | Purpose         |
|-------|-----------------|
| <h1>  | Main heading    |
| <h2>  | Subheading      |
| <p>   | Paragraph       |
| <br>  | Line break      |
| <hr>  | Horizontal line |
| <a>   | Hyperlink       |
| <img> | Image           |

| Tag      | Purpose        |
|----------|----------------|
| <ul>     | Unordered list |
| <ol>     | Ordered list   |
| <li>     | List item      |
| <table>  | Table          |
| <form>   | Form           |
| <input>  | Input field    |
| <button> | Button         |

## HTML Form

The basic structure of an HTML form involves the <form> element, which acts as a container for different input elements. Here is an example of a simple HTML form:

Example:



```
<form action="/submit_form" method="post">
<label for="fname">First name:</label><br>
<input type="text" id="fname" name="fname"><br>
<label for="lname">Last name:</label><br>
<input type="text" id="lname" name="lname"><br><br>
<input type="submit" value="Submit">
</form>
```

The user can enter information. Later, PHP will receive and process this information. So, the concept becomes:

HTML Form → User Input → PHP → Process Data → Database

## Common Form Elements:

| Type                    | Description                                                      |
|-------------------------|------------------------------------------------------------------|
| <input type="text">     | Displays a single-line text input field                          |
| <input type="radio">    | Displays a radio button (for selecting one of many choices)      |
| <input type="checkbox"> | Displays a checkbox (for selecting zero or more of many choices) |
| <input type="submit">   | Displays a submit button (for submitting the form)               |
| <input type="button">   | Displays a clickable button                                      |

## Introduction to PHP

PHP (Hypertext Preprocessor) is a server-side scripting language designed for web development but also used as a general-purpose programming language. It is widely used to create dynamic and interactive websites.

## Key Features

Server-Side Execution: PHP code runs on the server, and the output (usually HTML) is sent to the browser.

- Embedded in HTML: You can mix PHP code directly with HTML.
- Cross-Platform: Works on Windows, Linux, macOS, and more.
- Open Source: Free to use and supported by a large community.
- Database Integration: Works with MySQL, PostgreSQL, SQLite, and many others.
- Latest Version: As of 2025, PHP 8.4.8 is the latest stable release.

## Basic PHP Syntax:

A PHP script starts with `<? php` and ends with `?>`.

```
<?php // Simple PHP example
```

```
echo "Hello, World!";
```

```
?>
```

How it works:

1. The browser requests a .php file from the server.
2. The server passes the file to the PHP interpreter.
3. PHP executes the code and sends the resulting HTML back to the browser.

Why Use PHP?

- Easy to learn and widely supported.
- Excellent for form handling, session management, and file operations.
- Integrates seamlessly with HTML, CSS, JavaScript, and databases.
- Large ecosystem of frameworks (Laravel, CodeIgniter, Symfony).

## PHP Inside HTML

```
<!DOCTYPE html>
<html>
  <body>
    <center>
      <h2>
        <?php
          echo "This is PHP code inside html"
        ?>
      </h2>
    </center>
  </body>
</html>
```

## Introduction to MySQL

MySQL is an open-source Relational Database Management System (RDBMS) that uses Structured Query Language (SQL) to store, retrieve, and manage data efficiently.

It allows us to:

- create databases
- create tables
- insert data
- retrieve data
- update data
- delete data

```
-- Create a new database
CREATE DATABASE company_db;

-- Select the database
USE company_db;

-- Create a table
CREATE TABLE employees (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(100) NOT NULL,
  position VARCHAR(50),
  salary DECIMAL(10,2)
);

-- Insert data
INSERT INTO employees (name, position, salary)
VALUES ('Alice Johnson', 'Manager', 75000.00);

-- Retrieve data
SELECT * FROM employees;
```

## How MySQL Works

1. Client Request – Applications send SQL queries to the MySQL server.
2. SQL Parsing & Optimization – Queries are checked for syntax errors and optimized for performance.
3. Execution – The server interacts with the storage engine (e.g., InnoDB, MyISAM) to read/write data.
4. Result Delivery – The processed data is returned to the client.

## Why PHP + MySQL?

PHP handles the logic. MySQL handles the data.

PHP and MySQL are a powerful combination for creating dynamic and interactive web applications. MySQL is a popular database system used with PHP, known for its speed, reliability, and ease of use. It is ideal for both small and large applications and is widely used on the web.

## SETTING UP THE ENVIRONMENT:

Before writing and running PHP programs, we need a local development environment on our computer. PHP is a server-side language, so we cannot normally execute a .php file simply by opening it in a browser. We need a web server and a PHP interpreter to process the PHP code. Two commonly used packages for this are XAMPP and WAMP.

## What is XAMPP?

XAMPP is a free and open-source local development environment used to run PHP-based web applications on a computer.

The name XAMPP represents:

| Letter  | X              | A      | M             | P   | P    |
|---------|----------------|--------|---------------|-----|------|
| Meaning | Cross-Platform | Apache | MySQL/MariaDB | PHP | Perl |

For our PHP course, the most important components are: Apache + PHP + MySQL

Simple architecture

Browser → Apache Web Server → PHP → MySQL Database

## 2. What is WAMP?

WAMP is another local development environment mainly used on Windows for PHP development.

WAMP stands for:

| Letter  | W       | A      | M             | P   |
|---------|---------|--------|---------------|-----|
| Meaning | Windows | Apache | MySQL/MariaDB | PHP |

## XAMPP vs WAMP

| Feature           | XAMPP                                                                | WAMP                                |
|-------------------|----------------------------------------------------------------------|-------------------------------------|
| Full Form         | X = Cross-Platform, A = Apache, M = MySQL/MariaDB, P = PHP, P = Perl | Windows, Apache, MySQL/MariaDB, PHP |
| Operating System  | Windows, Linux, macOS                                                | Mainly Windows                      |
| Web Server        | Apache                                                               | Apache                              |
| PHP               | Yes                                                                  | Yes                                 |
| MySQL/MariaDB     | Yes                                                                  | Yes                                 |
| phpMyAdmin        | Usually included                                                     | Usually included                    |
| Beginner Friendly | Yes                                                                  | Yes                                 |
| Common Use        | PHP development and learning                                         | PHP development on Windows          |

## Steps for Installation of XAMPP

1. Download XAMPP: Download the appropriate XAMPP installer from the official Apache Friends website.

2. Run the Installer: Open the downloaded installer file and follow the installation wizard.
3. Select Components: Select the required components such as Apache, MySQL, PHP, and phpMyAdmin.
4. Choose Installation Directory: Select a suitable location for XAMPP. The default location is generally: C:\xampp
5. Complete Installation: Follow the instructions and click Next/Install to complete the installation.
6. Open XAMPP Control Panel: After installation, open the XAMPP Control Panel to manage the different services.
7. Start Apache: Start the Apache module to run the local web server and process PHP files.
8. Start MySQL: Start MySQL when the PHP application requires database connectivity.
9. Test the Installation:

Open a web browser and enter: <http://localhost/>

If the XAMPP dashboard appears, Apache has been installed and is running correctly.

10. Test phpMyAdmin:

Open: <http://localhost/phpmyadmin/>

This allows databases and tables to be created and managed through a graphical interface.

## Introduction to Apache, MySQL, and PHP

Apache, MySQL, and PHP are three important technologies commonly used together to develop dynamic and database-driven web applications. In a typical PHP development environment such as XAMPP,

Apache works as the web server, PHP handles server-side processing, and MySQL manages the application's data.

### Apache

Apache HTTP Server is an open-source web server software. Its main function is to receive requests from a web browser and provide the requested web resources or responses.

Main functions of Apache:

- Acts as a web server.
- Receives and processes HTTP requests.



- Serves HTML, CSS, images, and other web resources.
- Works with PHP to execute PHP applications.
- Provides a local server environment for development.

## MySQL

MySQL is an open-source Relational Database Management System (RDBMS) used to store and manage data. It organizes data into databases, tables, rows, and columns. PHP applications can communicate with MySQL to insert, retrieve, update, and delete information.

Main functions of MySQL:

- Stores application data.
- Organizes data into tables.
- Allows data to be inserted, retrieved, updated, and deleted.
- Supports SQL queries.
- Provides data storage for dynamic web applications.

## PHP

PHP (PHP: Hypertext Preprocessor) is an open-source server-side scripting language mainly used for web development. PHP code is executed on the server, and the resulting output is sent to the user's browser.

Example:

```
<?php  
echo "Welcome to PHP";  
?>
```

Output:

Welcome to PHP

Main functions of PHP:

- Processes server-side logic.
- Handles user input and forms.
- Creates dynamic webpages.
- Performs calculations and data processing.
- Connects web applications with databases such as MySQL.
- Handles sessions, authentication, and other backend operations.

## Configuring a Development Environment

A development environment is a set of software tools and configurations required to develop, test, debug, and run an application.

For PHP development, the environment must provide:

- A web server
- A PHP interpreter
- A database server
- A database management tool
- A code editor

Components of a PHP Development Environment

| Component     | Purpose                                             |
|---------------|-----------------------------------------------------|
| Code Editor   | Used to write and edit PHP code                     |
| Apache        | Acts as the web server                              |
| PHP           | Executes server-side PHP code                       |
| MySQL/MariaDB | Stores and manages application data                 |
| phpMyAdmin    | Provides a graphical interface to manage databases  |
| Browser       | Used to access and test the web application         |
| XAMPP/WAMP    | Provides a ready-made local development environment |

### Configuration Process:

Configuring a PHP development environment generally involves the following steps:

#### Step 1 — Install Required Software

Install a package such as:

- XAMPP
- WAMP

These provide the major components required for PHP development.

#### Step 2 — Configure the Web Server

The web server, commonly Apache, must be installed and configured to:

- Receive HTTP requests
- Serve website files
- Process PHP requests through PHP
- Provide access to local projects

### Step 3 — Configure PHP

PHP must be available to the web server so that .php files can be executed.

PHP configuration can be controlled through its configuration file: `php.ini`

This file contains settings related to:

- Error reporting
- Memory usage
- File uploads
- Execution time
- Extensions
- Date and time settings

### Step 4 — Configure the Database

A database server such as MySQL/MariaDB is configured to store application data.

PHP can communicate with the database using database extensions such as MySQLi or PDO.

### Step 5 — Configure the Web Root

A web root is the directory from which the web server serves website files.

In a typical XAMPP setup: `htdocs`

PHP projects are placed inside this directory.

```
htdocs
├── BUILD | BELIEVE | BECOME
├── myproject
│   └── index.php
```

### Step 6 — Configure Database Management

phpMyAdmin can be used to manage MySQL/MariaDB databases through a web interface.

It allows developers to:

- Create databases

- Create tables
- Insert records
- Update records
- Delete records
- Execute SQL queries

### Step 7 — Test the Environment

After configuration, each component should be tested.

- Apache → Web server working
- PHP → PHP code executing
- MySQL → Database working
- phpMyAdmin → Database management working
- Browser → Application accessible

### Localhost

A PHP development environment is often configured to run locally.

localhost refers to the computer on which the application is currently running.

For example: `http://localhost/`

A project may be accessed as: `http://localhost/myproject/`

### Port Configuration

A port is a logical communication endpoint used by network services.

Common default ports are:

| Service        | Typical Port |
|----------------|--------------|
| HTTP / Apache  | 80           |
| HTTPS / Apache | 443          |
| MySQL/MariaDB  | 3306         |

If another application is already using a required port, the corresponding service may fail to start. In such cases, the port configuration may need to be changed.

## Configuration of PHP Extensions

PHP functionality can be extended through PHP extensions.

For PHP + MySQL development, database-related extensions are important.

Examples: mysqli or PDO

Extensions can be enabled or configured through the PHP configuration.

## Why Configuration is Important

Proper configuration ensures that all components can communicate correctly.

If one component is incorrectly configured, the application may not work.

Examples:

- Apache not running → PHP page cannot be served through the local web server.
- PHP not configured → PHP code cannot be executed.
- MySQL not running → Database operations fail.
- Incorrect database configuration → PHP cannot connect to the database.
- Port conflict → A server may fail to start.

## Overall PHP Development Environment

